

Vidya Academy of Science & Technology

(A unit of Vidya International Charitable Trust)

Approved by AICTE & Affiliated to APJ Abdul Kalam Technological University



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

CSL201 DATA STRUCTURES LAB

LAB MANUAL

PREPARED BY:

Ms REMYA K R

APPROVED BY:

HOD, CSE

VISION OF THE INSTITUTION

Progress through education

MISSION OF THE INSTITUTION

To seek, strive for and scale greater heights of quality education

VISION OF THE DEPARTMENT

Progress through quality education and become a centre of excellence in computing knowledge

MISSION OF THE DEPARTMENT

To seek, to strive for advancements in technological and social aspects so as to create socially responsible engineers in the field of computer science, who will embark to further reduce gap between science and man-kind.

Program Educational Objectives (PEOs)

PEO-1: Graduates shall practice Computer Science and Engineering in the IT industry, public sector undertaking, as an entrepreneur or as a researcher pursuing a successful professional career.

PEO-2: Graduates shall demonstrate creativity, develop innovative ideas and work in teams to accomplish a common goal.

PEO-3: Graduates shall demonstrate project management skills, lifelong learning and social awareness upholding ethical values.

Program Specific Outcomes (PSOs)

PSO-1: The ability to apply the knowledge of computational structures and algorithms for solving complex engineering problems in multidisciplinary areas.

PSO-2: The ability to design, develop and evaluate computer based systems using advanced tools and technologies to meet the real world challenges.

Program Outcomes (POs)

Computer Science & Engineering graduates will be able to:

PO1:-Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and Computer Science & Engineering specialization to the solution of complex engineering problems.

PO2:-Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3:-Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4:-Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5:-Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6:-The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7:- Environment and sustainability: Understand the impact of the professional engineering solutions in

societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8:- Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice..

PO9:- Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary surroundings.

PO10:-Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11::Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12:-Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

CSL 201	DATA STRUCTURES LAB	CATEGORY	L	T	P	CREDIT	YEAR OF INTRODUCTION
		PCC	0	0	3	2	2019

Preamble: The aim of the Course is to give hands-on experience for Learners on creating and using different Data Structures. Data Structures are used to process data and arrange data in different formats for many applications. The most commonly performed operations on data structures are traversing, searching, inserting, deleting and a few special operations like merging and sorting.

Prerequisite: Topics covered under the course Programming in C (EST 102)

CO1	Write a time/space efficient program using arrays to provide necessary functionalities meeting a given set of user requirements
CO2	Write a time/space efficient program using linked lists to provide necessary functionalities meeting a given set of user requirements.
CO3	Write a time/space efficient program using graph to provide necessary functionalities meeting a given set of user requirements.
CO4	Write a time/space efficient program to sort a list of records based on a given key in the record
CO5	Design and implement an efficient data structure to represent given data.

Mapping of course outcomes with program outcomes

CO Number	PO STRENGTH												PSO STRENGTH	
	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12	PSO1	PSO2
CO1	2	3	3	1		2		2		3		2		2
CO2	2	3	3	3				2		2		2		2
CO3	2	3	3	2				2		3		3		2
CO4	2	3	3	1				2		2		2		2
CO5	2	3	3			2		2		1		2		2

Abstract POs defined by National Board of Accreditation			
PO#	Broad PO	PO#	Broad PO
PO1	Engineering Knowledge	PO7	Environment and Sustainability
PO2	Problem Analysis	PO8	Ethics
PO3	Design/Development of solutions	PO9	Individual and team work
PO4	Conduct investigations of complex problems	PO10	Communication
PO5	Modern tool usage	PO11	Project Management and Finance
PO6	The Engineer and Society	PO12	Life long learning

SYLLABUS

1. Implementation of Polynomials and Sparse matrices using arrays**
2. Implementation of Stack , Queues, Priority Queues, DEQUEUE and Circular Queues using arrays**
3. Application problems using stacks: Conversion of expression from one notation to another notation . **
4. Implementation of various linked list operations. **
5. Implementation of stack, queue and their applications using linked list.pression
6. Implementation of trees using linked list
7. Representation of polynomials using linked list, addition and multiplication of polynomials. **
8. Implementation of binary trees using linked lists and arrays- creations, insertion, deletion and traversal. **
9. Implementation of binary search trees – creation, insertion, deletion, search
10. Any application programs using trees
11. Implementation of sorting algorithms – bubble, insertion, selection, quick, merge sort and heap sort.**

12. Implementation of searching algorithms – linear search, binary search.**
13. Representation of graphs and computing various parameters (in degree, out degree etc.) - adjacency list, adjacency matrix.
14. Implementation of BFS and DFS for each graph representation.**
15. Implementation of hash table using your own mapping functions and observe collisions and overflow resolving schemes.**
16. Simulation of first-fit, best-fit and worst-fit allocations.
17. Simulation of a basic memory allocator and garbage collector using doubly linked list.
** mandatory.

DATA STRUCTURES LAB - PRACTICE QUESTIONS

1. Write a program to read two polynomials and store them in an array. Calculate the sum of the two polynomials and display the first polynomial, second polynomial and the resultant polynomial.
2. C Write a program to enter two matrices in normal form . Write a function to convert two matrices to tuple form and display it. Also find the transpose of the two matrices represented in tuple form and display it. Find the sum of the two matrices in tuple form and display the sum in tuple form.
3. Write a program to enter two matrices in normal form . Write a function to convert two matrices to tuple form and display it. Also find the transpose of the two matrices represented in tuple form and display it. Find the sum of the two matrices in tuple form and display the sum in tuple form.
4. Implement a circular queue using arrays with the operations:
 - 4.1. Insert an element to the queue.
 - 4.2. Delete an elements from the queue.
 - 4.3. Display the contents of the queue after each operation.
5. Implement a Queue using arrays with the operations:
 - 5.1. Insert elements to the Queue.
 - 5.2. Delete elements from the Queue.
 - 5.3. Display the contents of the Queue after each operation.
6. Implement a Stack using arrays with the operations:
 - 6.1. Pushing elements to the Stack.
 - 6.2. Popping elements from the Stack
 - 6.3. Display the contents of the Stack after each operation.

7. Implement a Priority Queue using arrays with the operations:
 - 7.1. Insert elements to the Priority Queue.
 - 7.2. Delete elements from the Priority Queue.
 - 7.3. Display the contents of the Priority Queue after each operation.
8. Implement a Double-Ended Queue (DEQUEUE) with the operations:
 - 8.1. Insert elements to the Front of the queue.
 - 8.2. Insert elements to the Rear of the queue
 - 8.3. Delete elements from the Front of the queue.
 - 8.4. Delete elements from the Rear of the queue.
 - 8.5. Display the queue after each operation.
9. Using stack convert an infix expression to a postfix expression and evaluate the postfix expression.
10. Write a program to convert an infix expression to a prefix expression using stacks.
11. Convert an infix expression to a postfix expression without using a stack
12. Write a menu driven program for performing the following operations on a Linked List:
 - 12.1. Display
 - 12.2. Insert at Beginning
 - 12.3. Insert at End
 - 12.4. Insert at a specified Position
 - 12.5. Delete from Beginning
 - 12.6. Delete from End
 - 12.7. Delete from a specified Position
13. Implement a stack using linked list with the operations:
 - 13.1. Push elements to the queue.
 - 13.2. Pop elements from the queue.
 - 13.3. Display the queue after each operation.
14. Implement a Queue using linked list with the operations:
 - 14.1. Insert an elements to the queue.
 - 14.2. Delete an elements from the queue.
 - 14.3. Display the queue after each operation.
15. Write a program to reverse the content of queue using stack
16. Write a program to read two polynomials and store them using linked list. Calculate the sum of the two polynomials and display the first polynomial, second polynomial and the resultant polynomial.

17. Write a program to read two polynomials and store them using linked list. Find the product of two polynomials and store the result using linked list. Display the resultant polynomial.
18. Write a program for addition of polynomials containing two variables using linked list.
19. The details of students(number, name, total-mark) are to be stored in a linked list. Write functions for the following operations:
 - 19.1. Insert
 - 19.2. Delete
 - 19.3. Search
 - 19.4. Sort on the basis of number
 - 19.5. Display the resultant list after every operation
20. Create a Doubly Linked List from a string taking each character from the string. Check if the given string is palindrome in an efficient method.
21. Create a binary tree with the following operations
 - 21.1. Insert a new node
 - 21.2. Inorder traversal.
 - 21.3. Preorder traversal.
 - 21.4. Postorder traversal.
 - 21.5. Delete a node.
22. Write a program to create a binary search tree and find the number of leaf nodes
23. Create a binary search tree with the following operations:
 - 23.1. Insert a new node .
 - 23.2. Inorder traversal.
 - 23.3. Preorder traversal.
 - 23.4. Postorder traversal.
 - 23.5. Delete a node.
24. Write a program to sort a set of numbers using a binary tree.
25. Represent any given graph and
 - 25.1. Perform a depth first search .
 - 25.2. Perform a breadth first search
26. Create a text file containing the name, height, weight of the students in a class. Perform Quick sort and Merge sort on this data and store the resultant data in two separate files. Also write the time

taken by the two sorting methods into the respective files.

Eg.	Sony Mathew	5.5	6
			0
	Arun Sajeev	5.7	5
			8
	Rajesh Kumar	6.1	7
			0

27. Write a program to sort a set of numbers using Heap sort and find a particular number from the sorted set using Binary Search.
28. Implement a Hash table using Chaining method. Let the size of hash table be 10 so that the index varies from 0 to 9.
29. Implement a Hash table that uses Linear Probing for collision resolution

CONTENTS

Expt. No.	Experiments	Page No
1	Polynomials addition using arrays	11
2	Transpose of a sparse matrix and addition of sparse matrices	15
3	Implementation of Stack , Queues, Priority Queues, DEQUEUE and Circular Queues using arrays	24
4	Conversion of Infix expression to postfix using stack	47
5	Stack and Queue using Linked list	53
6	Addition and multiplication of polynomials using singly linked lists	63
7	Implementation of binary trees using linked lists and arrays- creations, insertion, deletion and traversal.	70
8	binary search trees – creation, insertion, deletion, search	77
9	Implementation of sorting algorithms – bubble, insertion, selection, quick, merge sort and heap sort	84
10	searching algorithms – linear search, binary search	98
11	Implementation of BFS and DFS for each graph representation.	104
12	Simulation of a basic memory allocator and garbage collector using doubly linked list	110

EXP NO:1**POLYNOMIAL ADDITION USING ARRAY**

AIM: To Find the sum of two sparse polynomials using arrays

LEARNING OUTCOMES

Learners will be able to represent polynomials using arrays, store polynomial coefficients in arrays, perform polynomial addition by adding corresponding coefficients, implement a program for polynomial addition using arrays, display the resulting polynomial correctly, and analyze the efficiency of the algorithm.

DESCRIPTION

A sparse polynomial is a polynomial in which only a few coefficients are non-zero. Instead of storing all coefficients, the program stores only the non-zero terms as pairs of coefficients and exponents in arrays. During addition, the exponents of both polynomials are compared, and terms with the same exponent are added together while the remaining terms are copied to the result. This approach reduces memory usage and improves efficiency when working with sparse polynomials containing a large number of zero coefficients.

ALGORITHM

1.Initialize indices: $i = 0$ for polynomial A, $j = 0$ for polynomial B.

2.Compare exponents:

- If $A[i].exp > B[j].exp$, add $A[i]$ to result, increment i .
- If $A[i].exp < B[j].exp$, add $B[j]$ to result, increment j .
- If $A[i].exp == B[j].exp$, add coefficients $A[i].coeff + B[j].coeff$:

If $sum \neq 0$, add to the result.

Increment both i and j .

3.Copy remaining terms: If one polynomial has leftover terms, copy them directly to the result.

4.Print or return the result array.

PROGRAM

```
#include <stdio.h>
```

```
typedef struct {
```

```
    int coeff;
```

```
    int exp;
```

```
} Term;
```

```
int main() {
```

```
    Term poly1[20], poly2[20], result[40];
```

```
    int n1, n2, n3 = 0;
```

```
    int i, j;
```

```
    // Input for Polynomial 1
```

```
    printf("Enter number of terms in Polynomial 1: ");
```

```
    scanf("%d", &n1);
```

```
    printf("Enter terms (coefficient exponent) in decreasing order of exponent:\n");
```

```
    for (i = 0; i < n1; i++) {
```

```
        scanf("%d%d", &poly1[i].coeff, &poly1[i].exp);
```

```
    }
```

```
    // Input for Polynomial 2
```

```
    printf("Enter number of terms in Polynomial 2: ");
```

```
    scanf("%d", &n2);
```

```
    printf("Enter terms (coefficient exponent) in decreasing order of exponent:\n");
```

```
    for (i = 0; i < n2; i++) {
```

```
        scanf("%d%d", &poly2[i].coeff, &poly2[i].exp);
```

```
    }
```

```
    i = j = 0;
```

```
    // Merge-like addition
```

```
    while (i < n1 && j < n2) {
```

```
        if (poly1[i].exp == poly2[j].exp) {
```

```
            result[n3].coeff = poly1[i].coeff + poly2[j].coeff;
```

```
        result[n3].exp = poly1[i].exp;
        i++; j++; n3++;
    }
    else if (poly1[i].exp > poly2[j].exp) {
        result[n3] = poly1[i];
        i++; n3++;
    }
    else {
        result[n3] = poly2[j];
        j++; n3++;
    }
}

// Remaining terms of poly1
while (i < n1) {
    result[n3++] = poly1[i++];
}

// Remaining terms of poly2
while (j < n2) {
    result[n3++] = poly2[j++];
}

// Display Result
printf("\nResultant Polynomial:\n");
for (i = 0; i < n3; i++) {
    if (result[i].coeff != 0)
        printf("%dx^%d ", result[i].coeff, result[i].exp);
    if (i < n3 - 1)
        printf("+ ");
}
printf("\n");
return 0;
}
```

OUTPUT

Enter number of terms in Polynomial 1:3

Enter terms (coefficient exponent) in decreasing order of exponent:

3 4

2 2

5 0

$3x^4 + 2x^2 + 5$

Enter number of terms in Polynomial 2:3

Enter terms (coefficient exponent) in decreasing order of exponent:

4 3

2 2

7 0

$4x^3 + 2x^2 + 7$

Resultant Polynomial:

$3x^4 + 4x^3 + 4x^2 + 12x^0$

MODEL VIVA QUESTIONS

1. Why should the polynomial terms be arranged in descending order of exponents?
2. What happens if the exponents are equal?
3. What is the time and space complexity of polynomial addition?

RESULT

The sum of two sparse polynomials was successfully computed using arrays by comparing the exponents and adding the coefficients of like terms. The resulting polynomial was displayed correctly, demonstrating the efficient representation and addition of sparse polynomials using arrays.

EXP NO:2**TRANPOSE OF A SPARSE MATRIX AND ADDITION OF SPARSE MATRICES**

AIM:To Find the transpose of a sparse matrix and sum of two sparse matrices

LEARNING OUTCOMES

Learners will be able to represent sparse matrices using arrays, compute the transpose of a sparse matrix, perform the addition of two sparse matrices, implement these operations using array-based representations, display the resulting sparse matrices correctly, and analyze the efficiency of sparse matrix operations in terms of memory usage and execution time.

DESCRIPTION

This experiment demonstrates the implementation of the transpose of a sparse matrix and the addition of two sparse matrices using array representation. A sparse matrix contains mostly zero elements, so only the non-zero elements along with their row and column indices are stored in arrays, reducing memory usage. The transpose operation is performed by exchanging the row and column indices of each non-zero element, while the addition operation combines the non-zero elements of two sparse matrices by adding the values at corresponding positions. The experiment highlights the efficient storage, manipulation, and processing of sparse matrices using arrays, making matrix operations faster and more memory-efficient.

ALGORITHM**readSparse(sp)**

Input: Empty sparse matrix array sp[]

Output: Sparse matrix in 3-tuple form

1. Start
2. Read the number of rows $\rightarrow$ sp[0].row
3. Read the number of columns $\rightarrow$ sp[0].col
4. Read number of non-zero elements $\rightarrow$ sp[0].val
5. For i = 1 to sp[0].val
 - a. Read row, column, value

- b. Store them in sp[i].row, sp[i].col, sp[i].val
- 6. Stop

printSparse(sp)

Input: Sparse matrix in 3-tuple format

Output: Printed sparse matrix

- 1. Start
- 2. Let n = sp[0].val
- 3. Print heading (Row, Col, Value)
- 4. For i = 0 to n
 - a. Print sp[i].row, sp[i].col, sp[i].val
- 5. Stop

transpose(a, b)

Input: Sparse matrix a[]

Output: Transpose matrix b[]

- 1. Start
- 2. Set:
 - b[0].row = a[0].col
 - b[0].col = a[0].row
 - b[0].val = a[0].val
- 3. Initialize k = 1
- 4. For each column index i = 0 to a[0].col – 1
 - a. For each non-zero entry j = 1 to a[0].val
 - i. If a[j].col == i
 - Set b[k].row = a[j].col
 - Set b[k].col = a[j].row
 - Set b[k].val = a[j].val
 - Increment k
- 5. Stop

addSparse(a, b, c)**Input:** Sparse matrices A and B**Output:** Sparse matrix $C = A + B$

1. Start
2. If dimensions of A and B differ
→ Print “Addition not possible”
→ Stop
3. Initialize pointers:
 $i = 1$
 $j = 1$
 $k = 1$
4. Set $c[0].row = a[0].row$
5. Set $c[0].col = a[0].col$
6. Repeat while both matrices have elements:
 - a. If $a[i]$ and $b[j]$ have same (row, col):
 - Add values → $c[k].val = a[i].val + b[j].val$
 - Copy row, col
 - Increment i, j, k
 - b. Else if $a[i]$ comes earlier in row-major order
 - Copy $a[i]$ to $c[k]$
 - Increment i, k
 - c. Else
 - Copy $b[j]$ to $c[k]$
 - Increment j, k
7. Copy remaining entries of A (if any)
8. Copy remaining entries of B (if any)
9. Set $c[0].val = k - 1$
10. Stop

main()

1. Start

2. Declare A, B, T, C as sparse matrix arrays
3. Print "Enter Sparse Matrix A"
4. Call readSparse(A)
5. Print "Enter Sparse Matrix B"
6. Call readSparse(B)
7. Call transpose(A, T)
8. Call addSparse(A, B, C)
9. Print A using printSparse(A)
10. Print B using printSparse(B)
11. Print T using printSparse(T)
12. Print C using printSparse(C)
13. Stop

PROGRAM

```
#include <stdio.h>
#define MAX 50
// Structure for sparse matrix term (row, column, value)
typedef struct {
    int row, col, val;
} Term;
// Function to read sparse matrix
void readSparse(Term sp[]) {
    int i, r, c;
    printf("Enter number of rows and columns: ");
    scanf("%d%d", &sp[0].row, &sp[0].col);
    printf("Enter number of non-zero elements: ");
    scanf("%d", &sp[0].val);
    printf("Enter row, column and value of each non-zero element:\n");
    for (i = 1; i <= sp[0].val; i++) {
        scanf("%d%d%d", &sp[i].row, &sp[i].col, &sp[i].val);
    }
}
```

```
// Function to print sparse matrix in 3-tuple form
void printSparse(Term sp[]) {
    int i, n = sp[0].val;
    printf("\nRow\tCol\tValue\n");
    for (i = 0; i <= n; i++) {
        printf("%d\t%d\t%d\n", sp[i].row, sp[i].col, sp[i].val);
    }
}

// Function to find transpose
void transpose(Term a[], Term b[]) {
    int i, j, n = a[0].val;
    b[0].row = a[0].col;
    b[0].col = a[0].row;
    b[0].val = n;
    int k = 1;
    for (i = 0; i < a[0].col; i++) {
        for (j = 1; j <= n; j++) {
            if (a[j].col == i) {
                b[k].row = a[j].col;
                b[k].col = a[j].row;
                b[k].val = a[j].val;
                k++;
            }
        }
    }
}

// Function to add two sparse matrices
void addSparse(Term a[], Term b[], Term c[]) {
    if (a[0].row != b[0].row || a[0].col != b[0].col) {
        printf("Addition not possible. Dimensions mismatch!\n");
        return;
    }
}
```

```
}  
int i = 1, j = 1, k = 1;  
int n1 = a[0].val, n2 = b[0].val;  
c[0].row = a[0].row;  
c[0].col = a[0].col;  
while (i <= n1 && j <= n2) {  
    if (a[i].row == b[j].row && a[i].col == b[j].col) {  
        c[k].row = a[i].row;  
        c[k].col = a[i].col;  
        c[k].val = a[i].val + b[j].val;  
        i++; j++; k++;  
    }  
    else if (a[i].row < b[j].row ||  
             (a[i].row == b[j].row && a[i].col < b[j].col)) {  
        c[k++] = a[i++];  
    }  
    else {  
        c[k++] = b[j++];  
    }  
}  
while (i <= n1) c[k++] = a[i++];  
while (j <= n2) c[k++] = b[j++];  
c[0].val = k - 1;  
}  
int main() {  
    Term A[MAX], B[MAX], T[MAX], C[MAX];  
    // Read matrix A  
    printf("\nEnter Sparse Matrix A\n");  
    readSparse(A);  
    // Read matrix B  
    printf("\nEnter Sparse Matrix B\n");
```

```
readSparse(B);
// Transpose of A
transpose(A, T);
// Addition of A and B
addSparse(A, B, C);
// Display results
printf("\n--- Sparse Matrix A ---");
printSparse(A);
printf("\n--- Sparse Matrix B ---");
printSparse(B);
printf("\n--- Transpose of A ---");
printSparse(T);
printf("\n--- Sum A + B ---");
printSparse(C);
return 0;
}
```

OUTPUT

```
Enter Sparse Matrix A
Enter number of rows and columns: 3 3
Enter number of non-zero elements: 3
Enter row, column and value of each non-zero element:
0 2 5
1 1 3
2 0 4

Enter Sparse Matrix B
Enter number of rows and columns: 3 3
Enter number of non-zero elements: 3
Enter row, column and value of each non-zero element:
0 1 6
```

1 1 2

2 2 7

--- Sparse Matrix A ---

Row	Col	Value
-----	-----	-------

3	3	3
---	---	---

0	2	5
---	---	---

1	1	3
---	---	---

2	0	4
---	---	---

--- Sparse Matrix B ---

Row	Col	Value
-----	-----	-------

3	3	3
---	---	---

0	1	6
---	---	---

1	1	2
---	---	---

2	2	7
---	---	---

--- Transpose of A ---

Row	Col	Value
-----	-----	-------

3	3	3
---	---	---

2	0	5
---	---	---

1	1	3
---	---	---

0	2	4
---	---	---

--- Sum A + B ---

Row	Col	Value
-----	-----	-------

3	3	4
---	---	---

0	1	6
---	---	---

0	2	5
---	---	---

1	1	5
---	---	---

2	0	4
---	---	---

2	2	7
---	---	---

MODEL VIVA QUESTIONS

1. What is a sparse matrix?

2. Why do we use sparse matrix representation?
3. What are the advantages of sparse matrices?

RESULT

The transpose of the given sparse matrix and the sum of two sparse matrices were successfully obtained using array representation. The program correctly stored only the non-zero elements, performed the transpose by interchanging row and column indices, and computed the addition by combining elements with the same positions. The results verified the correctness and efficiency of array-based sparse matrix operations while minimizing memory usage.

EXP NO:3**STACK, QUEUE, DEQUEUE, AND CIRCULAR QUEUE USING ARRAY**

AIM: To Implement Stack, Queue, DEQUEUE, and Circular Queue using arrays.

LEARNING OUTCOMES

Learners will be able to implement stack, queue, deque (double-ended queue), and circular queue using arrays, perform insertion and deletion operations according to their respective principles (LIFO for stacks and FIFO for queues), manage overflow and underflow conditions, display the contents of each data structure, and analyze the efficiency of array-based implementations of stack and queue operations.

DESCRIPTION

A stack follows the **Last-In, First-Out (LIFO)** principle, where insertion (push) and deletion (pop) occur at the same end. A queue follows the **First-In, First-Out (FIFO)** principle, where elements are inserted at the rear and removed from the front. A deque allows insertion and deletion at both the front and rear ends, providing greater flexibility than a standard queue. A circular queue overcomes the limitation of a linear queue by treating the array as circular, enabling efficient reuse of vacant spaces created after deletions. This experiment helps in understanding the implementation, operations, and memory-efficient management of these linear data structures using arrays.

ALGORITHM**STACK****PUSH**

- step1. Check whether the TOP is $\geq \text{Size}-1$ then
- step2. Print that the “Stack is Full” and go to step 6
- step4. Else increment top by one
- step5. Store item to the location A[TOP]
- step6. Stop

POP

- step1. check whether the $TOP < 0$ or $p = ""$
- step2. Print that the "Stack is Empty" and go to step 5
- step3. Else Store the value of $A[TOP]$ to the variable item
- step4. Decrement top by one
- step5. Stop

DISPLAY

- step1. Check whether $TOP < 0$ or $p = ""$
- step2. Print that the "Stack is Empty"
- step3. Else Check whether the $TOP > SIZE - 1$ then Print that the "Stack is Full"
- step4. Else Print that the "Element at the Top of the stack is " $A[TOP]$
- step5. calculate $freespace = ((SIZE - 1) - TOP) / SIZE * 100$
- step6. Print calculated free space
- step7. Stop

LINEAR QUEUE USING ARRAYS

1. Repeat forever:
2. Display menu: Enqueue, Dequeue, Display, Exit
3. Read choice
4. If choice = 1 $\rightarrow$ Read x and call ENQUEUE(x)
 - If $rear == MAX - 1$, print **Overflow** and return.
 - If $front == -1$, set $front = 0$.
 - Increment rear.
 - Set $queue[rear] = x$.
 - Print that x is inserted
5. If choice = 2 $\rightarrow$ Call DEQUEUE()
 - If $front == -1$ or $front > rear$, print **Underflow** and return.
 - Print the element $queue[front]$.

Increment front

6. If choice = 3 → Call DISPLAY()

If front == -1 or front > rear, print **Queue empty** and return.

For i = front to rear, print queue[i]

7. If choice = 4 → Stop program

8. Else print **Invalid choice**

DEQUEUE (DOUBLE-ENDED QUEUE)

1. Repeat forever:

2. Display menu

3. Read user choice

4. if choice = 1 → Read x and call insertFront(x)

If deque is full

Condition: (front == 0 AND rear == MAX-1) OR (front == rear + 1)

→ Print **Deque Full** and return.

If deque is empty (front == -1)

→ Set front = rear = 0.

Else if front == 0

→ Set front = MAX - 1.

Else

→ Decrement front.

Store x at deque[front]

5. If choice = 2 → Read x and call insertRear(x)

If deque is full

→ Print **Deque Full** and return.

If deque is empty

→ Set front = rear = 0.

Else if rear == MAX - 1

→ Set rear = 0.

Else

→ Increment rear.

- Store x at deque[rear]
6. If choice = 3 → Call deleteFront()
If deque is empty (front == -1)
→ Print **Deque Empty** and return.
Print element at deque[front].
If front == rear
→ Set front = rear = -1.
Else if front == MAX - 1
→ Set front = 0.
Else
→ Increment front.
7. If choice = 4 → Call deleteRear()
If deque is empty
→ Print **Deque Empty** and return.
Print element at deque[rear].
If front == rear
→ Set front = rear = -1.
Else if rear == 0
→ Set rear = MAX - 1.
Else
→ Decrement rear
8. If choice = 5 → Call display()
If deque is empty
→ Print **Deque empty** and return.
Set i = front.
Repeat:
Print deque[i].
Set i = (i + 1) % MAX.
Until i == rear + 1 (mod MAX)
9. If choice = 6 → Exit program

10. Else print Invalid choice

CIRCULAR QUEUE

1. Repeat forever:
2. Display menu:
3. Read user choice
4. If choice = 1 → Read x and call ENQUEUE(x)

Check if queue is full:

Condition: (front == 0 AND rear == MAX-1) OR (rear + 1) % MAX == front

→ If TRUE, print Circular Queue Overflow and return.

If queue is empty (front == -1)

→ Set front = rear = 0.

Else

→ Update rear = (rear + 1) % MAX.

Insert element: cqueue[rear] = x.

Print x inserted.

5. If choice = 2 → Call DEQUEUE()

Check if queue is empty (front == -1)

→ If TRUE, print Circular Queue Underflow and return.

Print element at front: cqueue[front].

If front == rear

→ Only one element → Set front = rear = -1.

Else

→ Update front = (front + 1) % MAX.

6. If choice = 3 → Call DISPLAY()

Check if queue is empty (front == -1)

→ If TRUE, print Circular Queue is empty and return.

Set i = front.

Repeat:

Print cqueue[i]

If i == rear, stop

Else i = (i + 1) % MAX

7. If choice = 4 → Exit program
8. Else → Print Invalid choice

PROGRAM**STACK**

```
#include<stdio.h>
#include<stdlib.h>
int a[20];
int size=20;
main()
{ int top= -1,choice,item,c,i;
do

{
puts("Press the operation to perform\n1:Push\n2:Pop\n3:Display"); scanf("%d",&choice);
switch(choice)
{ case 1: if (top>=size-1)

{ printf("Stack overflow");
}
else
{ printf("Enter the element to add\n");

scanf("%d",&item);
top+=1; a[top]=item;
printf("Element is
pushed:%d\n",item); break;

}
case 2: if(top<0)
puts("Stack underflow");
```

```
else
{
item=a[top];
top-=1;
printf("Element is popped:%d\n",item);
break;
}
case 3: if (top>=size-1)

{printf("Stack overflow");
break;}
if(top<0)
{puts("Stack underflow");
break;}
for(i=0;i<top+1;i++)
{printf("%d\t",a[i]);}
}
}
```

LINEAR QUEUE USING ARRAYS

```
#include <stdio.h>
#define MAX 5

int queue[MAX];
int front = -1, rear = -1;

void enqueue(int x) {
    if (rear == MAX - 1)
        printf("Queue Overflow!\n");
    else {
        if (front == -1) front = 0;
        queue[++rear] = x;
        printf("%d inserted.\n", x);
    }
}
```

```
    }  
}  
  
void dequeue() {  
    if (front == -1 || front > rear)  
        printf("Queue Underflow!\n");  
    else  
        printf("%d deleted.\n", queue[front++]);  
}  
  
void display() {  
    if (front == -1 || front > rear)  
        printf("Queue is empty.\n");  
    else {  
        printf("Queue: ");  
        for (int i = front; i <= rear; i++)  
            printf("%d ", queue[i]);  
        printf("\n");  
    }  
}  
  
int main() {  
    int choice, x;  
  
    while (1) {  
        printf("\n--- LINEAR QUEUE MENU ---\n");  
        printf("1. Enqueue\n2. Dequeue\n3. Display\n4. Exit\n");  
        printf("Enter choice: ");  
        scanf("%d", &choice);  
  
        switch (choice) {
```

```
case 1: printf("Enter value: "); scanf("%d", &x); enqueue(x); break;
case 2: dequeue(); break;
case 3: display(); break;
case 4: return 0;
default: printf("Invalid choice!\n");
}
}
}
```

DEQUEUE (DOUBLE-ENDED QUEUE)

```
#include <stdio.h>
#define MAX 5
int deque[MAX];
int front = -1, rear = -1;
void insertFront(int x) {
    if ((front == 0 && rear == MAX - 1) || (front == rear + 1))
        printf("Deque is Full!\n");
    else {
        if (front == -1)
            front = rear = 0;
        else if (front == 0)
            front = MAX - 1;
        else
            front--;
        deque[front] = x;
        printf("%d inserted at front.\n", x);
    }
}
void insertRear(int x) {
    if ((front == 0 && rear == MAX - 1) || (front == rear + 1))
        printf("Deque is Full!\n");
    else {
```

```
    if (rear == -1)
        front = rear = 0;
    else if (rear == MAX - 1)
        rear = 0;
    else
        rear++;
    deque[rear] = x;
    printf("%d inserted at rear.\n", x);
}
}
void deleteFront() {
    if (front == -1)
        printf("Deque is Empty!\n");
    else {
        printf("%d deleted from front.\n", deque[front]);
        if (front == rear)
            front = rear = -1;
        else if (front == MAX - 1)
            front = 0;
        else
            front++;
    }
}
void deleteRear() {
    if (rear == -1)
        printf("Deque is Empty!\n");
    else {
        printf("%d deleted from rear.\n", deque[rear]);
        if (front == rear)
            front = rear = -1;
        else if (rear == 0)
            rear = MAX - 1;
    }
}
```

```
        rear = MAX - 1;
    else
        rear--;
    }
}

void display() {
    if (front == -1)
        printf("Deque is empty.\n");
    else {
        printf("Deque: ");
        int i = front;
        while (1) {
            printf("%d ", deque[i]);
            if (i == rear) break;
            i = (i + 1) % MAX;
        }
        printf("\n");
    }
}

int main() {
    int choice, x;
    while (1) {
        printf("\n--- DEQUEUE MENU ---\n");
        printf("1. Insert Front\n2. Insert Rear\n3. Delete Front\n4. Delete Rear\n5. Display\n6. Exit\n");
        scanf("%d", &choice);
        switch (choice) {
            case 1: printf("Enter value: "); scanf("%d", &x); insertFront(x); break;
            case 2: printf("Enter value: "); scanf("%d", &x); insertRear(x); break;
            case 3: deleteFront(); break;
            case 4: deleteRear(); break;
            case 5: display(); break;
        }
    }
}
```

```
        case 6: return 0;
        default: printf("Invalid choice!\n");
    }
}
```

CIRCULAR QUEUE

```
#include <stdio.h>
#define MAX 5
int cqueue[MAX];
int front = -1, rear = -1;
void enqueue(int x) {
    if ((front == 0 && rear == MAX - 1) || (rear + 1) % MAX == front)
        printf("Circular Queue Overflow!\n");
    else {
        if (front == -1)
            front = rear = 0;
        else
            rear = (rear + 1) % MAX;
        cqueue[rear] = x;
        printf("%d inserted.\n", x);
    }
}
void dequeue() {
    if (front == -1)
        printf("Circular Queue Underflow!\n");
    else {
        printf("%d deleted.\n", cqueue[front]);
        if (front == rear)
            front = rear = -1;
        else
            front = (front + 1) % MAX;
    }
}
```

```
    }  
}  
void display() {  
    if (front == -1)  
        printf("Circular Queue is empty.\n");  
    else {  
        printf("Circular Queue: ");  
        int i = front;  
        while (1) {  
            printf("%d ", cqueue[i]);  
            if (i == rear) break;  
            i = (i + 1) % MAX;  
        }  
        printf("\n");  
    }  
}  
int main() {  
    int choice, x;  
    while (1) {  
        printf("\n--- CIRCULAR QUEUE MENU ---\n");  
        printf("1. Enqueue\n2. Dequeue\n3. Display\n4. Exit\n");  
        scanf("%d", &choice);  
        switch (choice) {  
            case 1: printf("Enter value: "); scanf("%d", &x); enqueue(x); break;  
            case 2: dequeue(); break;  
            case 3: display(); break;  
            case 4: return 0;  
            default: printf("Invalid choice!\n");  
        }  
    }  
}
```

OUTPUT**STACK**

Enter the size of stack 5

1. push
2. pop
3. display
4. exit

Enter choice:1

Enter theValue 2

Enter choice :1

Enter the Value 5

Enter choice :3

2 5

Enter choice :2

5 popped

Enter choice :2

2 popped

LINEAR QUEUE USING ARRAYS

--- LINEAR QUEUE MENU ---

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter choice: 1

Enter value: 10

10 inserted.

--- LINEAR QUEUE MENU ---

1. Enqueue
2. Dequeue

3. Display

4. Exit

Enter choice: 1

Enter value: 20

20 inserted.

--- LINEAR QUEUE MENU ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter choice: 1

Enter value: 30

30 inserted.

--- LINEAR QUEUE MENU ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter choice: 3

Queue: 10 20 30

--- LINEAR QUEUE MENU ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter choice: 2

10 deleted.

--- LINEAR QUEUE MENU ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter choice: 3

Queue: 20 30

--- LINEAR QUEUE MENU ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter choice: 1

Enter value: 40

40 inserted.

--- LINEAR QUEUE MENU ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter choice: 1

Enter value: 50

50 inserted.

--- LINEAR QUEUE MENU ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter choice: 1

Enter value: 60

Queue Overflow!

--- LINEAR QUEUE MENU ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter choice: 3

Queue: 20 30 40 50

--- LINEAR QUEUE MENU ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter choice: 4

DEQUEUE (DOUBLE-ENDED QUEUE)

--- DEQUEUE MENU ---

1. Insert Front

2. Insert Rear

3. Delete Front

4. Delete Rear

5. Display

6. Exit

Enter choice: 2

Enter value: 10

10 inserted at rear.

--- DEQUEUE MENU ---

1. Insert Front
2. Insert Rear
3. Delete Front
4. Delete Rear
5. Display
6. Exit

Enter choice: 2

Enter value: 20

20 inserted at rear.

--- DEQUEUE MENU ---

1. Insert Front
2. Insert Rear
3. Delete Front
4. Delete Rear
5. Display
6. Exit

Enter choice: 1

Enter value: 5

5 inserted at front.

--- DEQUEUE MENU ---

1. Insert Front
2. Insert Rear
3. Delete Front
4. Delete Rear
5. Display
6. Exit

Enter choice: 5

Deque: 5 10 20

--- DEQUEUE MENU ---

1. Insert Front
2. Insert Rear
3. Delete Front
4. Delete Rear
5. Display
6. Exit

Enter choice: 3

5 deleted from front.

--- DEQUEUE MENU ---

1. Insert Front
2. Insert Rear
3. Delete Front
4. Delete Rear
5. Display
6. Exit

Enter choice: 4

20 deleted from rear.

--- DEQUEUE MENU ---

1. Insert Front
2. Insert Rear
3. Delete Front
4. Delete Rear
5. Display
6. Exit

Enter choice: 5

Deque: 10

--- DEQUEUE MENU ---

1. Insert Front
2. Insert Rear
3. Delete Front
4. Delete Rear
5. Display
6. Exit

Enter choice: 6

CIRCULAR QUEUE

--- CIRCULAR QUEUE MENU ---

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter choice: 1

Enter value: 10

10 inserted.

--- CIRCULAR QUEUE MENU ---

1. Enqueue
2. Dequeue
3. Display
4. Exit

Enter choice: 1

Enter value: 20

20 inserted.

--- CIRCULAR QUEUE MENU ---

1. Enqueue
2. Dequeue

3. Display

4. Exit

Enter choice: 1

Enter value: 30

30 inserted.

--- CIRCULAR QUEUE MENU ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter choice: 3

Circular Queue: 10 20 30

--- CIRCULAR QUEUE MENU ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter choice: 2

10 deleted.

--- CIRCULAR QUEUE MENU ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter choice: 1

Enter value: 40

40 inserted.

--- CIRCULAR QUEUE MENU ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter choice: 3

Circular Queue: 20 30 40

--- CIRCULAR QUEUE MENU ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter choice: 1

Enter value: 50

50 inserted.

--- CIRCULAR QUEUE MENU ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter choice: 1

Enter value: 60

Circular Queue Overflow!

--- CIRCULAR QUEUE MENU ---

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter choice: 3

Circular Queue: 20 30 40 50

--- CIRCULAR QUEUE MENU ---

1. Enqueue
2. Dequeue
3. Display
4. Exit

MODEL VIVA QUESTIONS

1. Which principle does a stack and queue follow?
2. What is the time complexity of Push and Pop?
3. What is an Input Restricted Deque?

RESULT

The implementation of Stack, Queue, Deque (Double-Ended Queue), and Circular Queue using arrays was successfully completed and verified the correctness and efficiency of array-based implementations of these linear data structures.

EXP NO:4**INFIX EXPRESSION TO POSTFIX (OR PREFIX) AND EVALUATION**

AIM: To Convert infix expression to postfix (or prefix) and then evaluate using stack

LEARNING OUTCOMES

Learners will be able to convert infix expressions into postfix and prefix expressions using stack operations, evaluate postfix and prefix expressions correctly, apply operator precedence and associativity rules during expression conversion, implement stack-based algorithms for expression processing, and analyze the efficiency of expression conversion and evaluation techniques.

DESCRIPTION

An infix expression places operators between operands and requires operator precedence and associativity rules for correct evaluation. By using a stack, the infix expression is efficiently converted into postfix or prefix notation, eliminating the need for parentheses during evaluation. The postfix and prefix expressions are then evaluated using stack operations to obtain the final result. This experiment helps in understanding stack applications in expression conversion, operator precedence handling, and efficient expression evaluation.

ALGORITHM

1. Start
2. Declare infix and postfix strings
3. Read infix expression from user
4. Call **infixToPostfix()** to convert infix to postfix

Initialize an empty stack

Scan the infix expression from left to right

For each character **ch** in the infix expression:

Case 1: If **ch** is an operand

→ Append it to the postfix expression

Case 2: If **ch** is '('

→ Push it onto the stack

Case 3: If `ch` is `)`

→ Repeatedly pop from the stack and append to postfix

→ Stop when `(` is encountered

→ Remove `(` from stack

Case 4: If `ch` is an operator

→ While the stack is not empty **AND**

$\text{precedence}(\text{top of stack}) \geq \text{precedence}(\text{ch})$

→ Pop from stack and append to postfix

→ Push `ch` onto the stack

After the full expression is scanned:

→ Pop all remaining operators from the stack into postfix

Add NULL terminator to postfix

5. Print the postfix expression

6. Call **evaluatePostfix()**

Initialize empty evaluation stack

Scan the postfix expression from left to right

For each symbol `ch`:

Case 1: If `ch` is a digit

→ Convert character to number

→ Push it onto evaluation stack

Case 2: If `ch` is an operator

→ Pop operand2

→ Pop operand1

→ Apply the operator: `+`, `-`, `*`, `/`, `^`

→ Push result back onto stack

After scanning ends:

→ Pop the final value from stack → This is the result

7. Print the evaluated result

8. Stop

PROGRAM

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>

#define MAX 100
char stack[MAX];
int top = -1;
void push(char x) {
    stack[++top] = x;
}
char pop() {
    if (top == -1)
        return -1;
    else
        return stack[top--];
}
int precedence(char x) {
    if (x == '(') return 0;
    if (x == '+' || x == '-') return 1;
    if (x == '*' || x == '/') return 2;
    if (x == '^') return 3;
    return 0;
}
int isOperator(char x) {
    return (x == '+' || x == '-' || x == '*' || x == '/' || x == '^');
}
// ----- INFIX TO POSTFIX -----
void infixToPostfix(char infix[], char postfix[]) {
    int i, j = 0;
```

```
char x;
for (i = 0; infix[i] != '\0'; i++) {
    if (isalnum(infix[i])) {
        postfix[j++] = infix[i];
    }
    else if (infix[i] == '(') {
        push(infix[i]);
    }
    else if (infix[i] == ')') {
        while ((x = pop()) != '(')
            postfix[j++] = x;
    }
    else {
        while (precedence(stack[top]) >= precedence(infix[i]))
            postfix[j++] = pop();
        push(infix[i]);
    }
}
while (top != -1)
    postfix[j++] = pop();
postfix[j] = '\0';
}

// ----- POSTFIX EVALUATION -----

int evalStack[MAX];
int etop = -1;
void epush(int x) {
    evalStack[++etop] = x;
}
int epop() {
    return evalStack[etop--];
}
```

```
int evaluatePostfix(char postfix[]) {
    int i, op1, op2;
    for (i = 0; postfix[i] != '\0'; i++) {
        if (isdigit(postfix[i])) {
            epush(postfix[i] - '0'); // convert char to int
        }
        else if (isOperator(postfix[i])) {
            op2 = epop();
            op1 = epop();
            switch (postfix[i]) {
                case '+': epush(op1 + op2); break;
                case '-': epush(op1 - op2); break;
                case '*': epush(op1 * op2); break;
                case '/': epush(op1 / op2); break;
                case '^': {
                    int p = 1;
                    for (int k = 0; k < op2; k++) p *= op1;
                    epush(p);
                    break;
                }
            }
        }
    }
    return epop();
}

// ----- MAIN -----

int main() {
    char infix[MAX], postfix[MAX];

    printf("Enter infix expression: ");
    scanf("%s", infix);
    infixToPostfix(infix, postfix);
}
```

```
printf("\nPostfix Expression: %s\n", postfix);  
int result = evaluatePostfix(postfix);  
printf("Evaluated Result: %d\n", result);  
return 0;  
}
```

OUTPUT

Enter infix expression: 5*(6-2)

Postfix Expression: 562-*

Evaluated Result: 20

Enter infix expression: 8/4+3*2

Postfix Expression: 84/32*+

Evaluated Result: 8

MODEL VIVA QUESTIONS

1. What is operator precedence?
2. What is the time complexity of infix to postfix conversion?
3. Which notation is used by computers for expression evaluation?
4. How is a prefix expression evaluated?

RESULT

The infix expression was successfully converted into postfix (or prefix) notation using stack operations, and the converted expression was evaluated correctly and verified the effectiveness of stack-based algorithms for expression conversion and evaluation.

EXP NO:5**STACK AND QUEUE USING SINGLY LINKED LIST**

AIM: Implement stack and queue using singly linked lists

LEARNING OUTCOMES

learners will be able to implement stack and queue using a singly linked list, perform insertion and deletion operations according to the **LIFO** principle for stacks and the **FIFO** principle for queues, dynamically manage memory using linked list nodes, handle overflow and underflow conditions, display the contents of the data structures, and analyze the efficiency of linked list-based implementations of stack and queue operations.

DESCRIPTION

In a linked list implementation, each node contains a data element and a pointer to the next node, allowing dynamic memory allocation without requiring a fixed-size array. A **Stack** follows the **Last-In, First-Out (LIFO)** principle, where insertion (push) and deletion (pop) are performed at the beginning of the linked list. A **Queue** follows the **First-In, First-Out (FIFO)** principle, where insertion (enqueue) is performed at the rear and deletion (dequeue) is carried out from the front. This experiment demonstrates how linked lists provide efficient insertion and deletion operations while eliminating the limitations of fixed-size array implementations.

ALGORITHM**STACK****PUSH Operation**

1. Start
2. Create a new node newNode
3. If newNode == NULL
 Print "**Stack Overflow**"
 Stop
4. Assign value to newNode->data
5. Set newNode->next = top

6. Set top = newNode
7. Stop

POP Operation

1. Start
2. If top == NULL
 Print "**Stack Underflow**"
 Stop
3. Store top in temporary pointer temp
4. Print temp->data
5. Set top = temp->next
6. Free temp
7. Stop

DISPLAY Operation

1. Start
2. If top == NULL
 Print "**Stack is empty**"
 Stop
3. Set temp = top
4. While temp != NULL
 Print temp->data
 Move temp = temp->next
5. Stop

QUEUE**ENQUEUE Operation**

1. Start
2. Create a new node newNode
3. If newNode == NULL
 Print "**Queue Overflow**"

Stop

4. Assign value to newNode->data
5. Set newNode->next = NULL
6. If front == NULL
Set front = rear = newNode
7. Else
Set rear->next = newNode
Set rear = newNode
8. Stop

DEQUEUE Operation

1. Start
2. If front == NULL
Print "**Queue Underflow**"
Stop
3. Store front in temporary pointer temp
4. Print temp->data
Set front = front->next
5. If front == NULL
Set rear = NULL
6. Free temp
7. Stop

DISPLAY Operation

1. If front == NULL
Print "**Queue is empty**"
Stop
2. Set temp = front
3. While temp != NULL
Print temp->data
Move temp = temp->next

4. Stop

PROGRAM

STACK

```
#include <stdio.h>
#include <stdlib.h>
struct node {
    int data;
    struct node *next;
};
struct node *top = NULL;
/* Push operation */
void push(int value) {
    struct node *newNode = (struct node *)malloc(sizeof(struct node));
    if (newNode == NULL) {
        printf("Stack Overflow\n");
        return;
    }
    newNode->data = value;
    newNode->next = top;
    top = newNode;
}
/* Pop operation */
void pop() {
    if (top == NULL) {
        printf("Stack Underflow\n");
        return;
    }
    struct node *temp = top;
    printf("Popped element: %d\n", temp->data);
    top = top->next;
```

```
    free(temp);
}
/* Display stack */
void displayStack() {
    struct node *temp = top;
    if (temp == NULL) {
        printf("Stack is empty\n");
        return;
    }
    printf("Stack elements: ");
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}
```

QUEUE

```
#include <stdio.h>
#include <stdlib.h>
struct node {
    int data;
    struct node *next;
};
struct node *front = NULL;
struct node *rear = NULL;
/* Enqueue operation */
void enqueue(int value) {
    struct node *newNode = (struct node *)malloc(sizeof(struct node));
    if (newNode == NULL) {
        printf("Queue Overflow\n");
        return;
    }
}
```

```
}
newNode->data = value;
newNode->next = NULL;

if (front == NULL) {
    front = rear = newNode;
} else {
    rear->next = newNode;
    rear = newNode;
}
}

/* Dequeue operation */
void dequeue() {
    if (front == NULL) {
        printf("Queue Underflow\n");
        return;
    }
    struct node *temp = front;
    printf("Dequeued element: %d\n", temp->data);
    front = front->next;

    if (front == NULL)
        rear = NULL;
    free(temp);
}

/* Display queue */
void displayQueue() {
    struct node *temp = front;
    if (temp == NULL) {
        printf("Queue is empty\n");
        return;
    }
}
```

```
}  
printf("Queue elements: ");  
while (temp != NULL) {  
    printf("%d ", temp->data);  
    temp = temp->next;  
}  
printf("\n");  
}
```

OUTPUT

STACK USING LINKED LIST

1. Push
2. Pop
3. Display
4. Exit

Enter your choice: 1

Enter element to push: 10

Element pushed into stack

Enter your choice: 1

Enter element to push: 20

Element pushed into stack

Enter your choice: 1

Enter element to push: 30

Element pushed into stack

Enter your choice: 3

Stack elements: 30 20 10

Enter your choice: 2

Popped element: 30

Enter your choice: 3

Stack elements: 20 10

Enter your choice: 2

Popped element: 20

Enter your choice: 2

Popped element: 10

Enter your choice: 2

Stack Underflow

Enter your choice: 4

Exit

QUEUE USING LINKED LIST

1. Enqueue

2. Dequeue

3. Display

4. Exit

Enter your choice: 1

Enter element to enqueue: 10

Element inserted into queue

Enter your choice: 1

Enter element to enqueue: 20

Element inserted into queue

Enter your choice: 1

Enter element to enqueue: 30

Element inserted into queue

Enter your choice: 3

Queue elements: 10 20 30

Enter your choice: 2

Dequeued element: 10

Enter your choice: 3

Queue elements: 20 30

Enter your choice: 2

Dequeued element: 20

Enter your choice: 2

Dequeued element: 30

Enter your choice: 2

Queue Underflow

Enter your choice: 4

Exit

MODEL VIVA QUESTIONS

- 1.How is memory allocated for a linked list?
- 2.What are the basic operations of a linked list?
- 3.What is the advantage of a doubly linked list?
- 4.Which memory is used for linked lists?

RESULT

The implementation of Stack and Queue using a Singly Linked List was successfully completed. The stack correctly performed push, pop, and display operations following the LIFO principle, while the queue correctly performed enqueue, dequeue, and display operations following the FIFO principle.

EXP NO:6**POLYNOMIAL ADDITION AND MULTIPLICATION USING SINGLY LINKED LIST**

AIM: Implement addition and multiplication of polynomials using singly linked lists

LEARNING OUTCOMES

learners will be able to represent polynomials using a singly linked list, perform polynomial addition and multiplication by manipulating linked list nodes, combine like terms efficiently, dynamically manage polynomial terms using linked lists, display the resulting polynomial in standard form, and analyze the efficiency of linked list-based polynomial operations.

DESCRIPTION

Each node in the linked list represents a term of the polynomial, containing the coefficient, exponent, and a pointer to the next node. Polynomial addition is performed by comparing the exponents of corresponding terms and combining coefficients of like terms, while polynomial multiplication is carried out by multiplying each term of one polynomial with every term of the other polynomial and then combining terms with the same exponent. The use of a singly linked list allows dynamic memory allocation, efficient storage of polynomial terms, and easy manipulation of polynomials with varying numbers of terms.

ALGORITHM

1. start
2. Create two polynomials poly1 and poly2 using CREATE POLYNOMIAL.
3. Display poly1 and poly2.
4. Call ADD POLYNOMIALS(poly1, poly2) → store in sum.
5. Display sum.
6. Call MULTIPLY POLYNOMIALS(poly1, poly2) → store in product.
7. Display product

Node Structure

- Each node represents a term of the polynomial:
 - coeff $\rightarrow$ coefficient of term
 - exp $\rightarrow$ exponent of term
 - next $\rightarrow$ pointer to next term
- Maintain **head pointer** for each polynomial.

INSERT TERM (poly, coeff, exp)

1. Create a new node newNode with coeff and exp.
2. If list is empty (poly == NULL)
 - $\rightarrow$ Make newNode the head of list.
3. Else, traverse the list to find correct position based on **descending exponent**.
4. If a node with same exponent exists
 - $\rightarrow$ Add coefficients (existing.coeff += coeff).
5. Else
 - $\rightarrow$ Insert newNode at proper position.

ADDITION OF TWO POLYNOMIALS (poly1, poly2)

1. Initialize result = NULL.
2. Traverse poly1, for each term:
 - $\rightarrow$ INSERT TERM(result, coeff, exp).
3. Traverse poly2, for each term:
 - $\rightarrow$ INSERT TERM(result, coeff, exp).
4. Return result.

MULTIPLICATION OF TWO POLYNOMIALS (poly1, poly2)

1. Initialize result = NULL.
2. For each term t1 in poly1:
 - a. For each term t2 in poly2:
 - $\rightarrow$ Multiply coefficients: $c = t1.coeff * t2.coeff$
 - $\rightarrow$ Add exponents: $e = t1.exp + t2.exp$

→ INSERT TERM(result, c, e).

3. Return result.

DISPLAY POLYNOMIAL (poly)

1. If poly == NULL
→ Print 0 and return.
2. Traverse the list:
→ Print each term as coeff x^{exp}.
3. Separate terms with +.

CREATE POLYNOMIAL

1. Read number of terms n.
2. Repeat n times:
→ Read coeff and exp.
→ Call INSERT TERM(poly, coeff, exp).

PROGRAM

```
#include <stdio.h>
#include <stdlib.h>
// Node structure for polynomial term
typedef struct Node {
    int coeff;
    int exp;
    struct Node* next;
} Node;
// Function to create a new term
Node* createNode(int coeff, int exp) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->coeff = coeff;
    newNode->exp = exp;
    newNode->next = NULL;
    return newNode;
}
```

```
}  
// Function to insert term in descending order of exponent  
void insertTerm(Node** poly, int coeff, int exp) {  
    Node* newNode = createNode(coeff, exp);  
    if (*poly == NULL || exp > (*poly)->exp) {  
        newNode->next = *poly;  
        *poly = newNode;  
        return;  
    }  
    Node* temp = *poly;  
    while (temp->next != NULL && temp->next->exp >= exp) {  
        if (temp->next->exp == exp) { // Same exponent, add coefficients  
            temp->next->coeff += coeff;  
            free(newNode);  
            return;  
        }  
        temp = temp->next;  
    }  
    newNode->next = temp->next;  
    temp->next = newNode;  
}  
// Function to display polynomial  
void displayPoly(Node* poly) {  
    if (poly == NULL) {  
        printf("0\n");  
        return;  
    }  
    Node* temp = poly;  
    while (temp != NULL) {  
        printf("%dx^%d", temp->coeff, temp->exp);  
        temp = temp->next;  
    }
```

```
    if (temp != NULL) printf(" + ");
}
printf("\n");
}

// Function to add two polynomials
Node* addPoly(Node* p1, Node* p2) {
    Node* result = NULL;
    while (p1 != NULL) {
        insertTerm(&result, p1->coeff, p1->exp);
        p1 = p1->next;
    }
    while (p2 != NULL) {
        insertTerm(&result, p2->coeff, p2->exp);
        p2 = p2->next;
    }
    return result;
}

// Function to multiply two polynomials
Node* multiplyPoly(Node* p1, Node* p2) {
    Node* result = NULL;
    for (Node* t1 = p1; t1 != NULL; t1 = t1->next) {
        for (Node* t2 = p2; t2 != NULL; t2 = t2->next) {
            insertTerm(&result, t1->coeff * t2->coeff, t1->exp + t2->exp);
        }
    }
    return result;
}

// Function to create polynomial by user input
Node* createPolynomial() {
    int n, coeff, exp;
```

```
Node* poly = NULL;
printf("Enter number of terms: ");
scanf("%d", &n);
for (int i = 0; i < n; i++) {
    printf("Enter coefficient and exponent for term %d: ", i+1);
    scanf("%d %d", &coeff, &exp);
    insertTerm(&poly, coeff, exp);
}
return poly;
}
// Main function
int main() {
    printf("Polynomial 1:\n");
    Node* poly1 = createPolynomial();
    printf("Polynomial 2:\n");
    Node* poly2 = createPolynomial();
    printf("\nPolynomial 1: ");
    displayPoly(poly1);
    printf("Polynomial 2: ");
    displayPoly(poly2);
    Node* sum = addPoly(poly1, poly2);
    printf("\nSum: ");
    displayPoly(sum);
    Node* product = multiplyPoly(poly1, poly2);
    printf("Product: ");
    displayPoly(product);
    return 0;
}
```

OUTPUT

Polynomial 1:

Number of terms: 3

Terms: (3 2), (2 1), (1 0)

Polynomial 2:

Number of terms: 2

Terms: (4 1), (5 0)

Polynomial 1: $3x^2 + 2x^1 + 1x^0$

Polynomial 2: $4x^1 + 5x^0$

Sum: $3x^2 + 6x^1 + 6x^0$

Product: $12x^3 + 23x^2 + 14x^1 + 5x^0$

MODEL VIVA QUESTIONS

1. Why is a singly linked list used for polynomial representation?
2. How is polynomial addition performed?
3. What are the advantages of linked lists over arrays for polynomial operations?
4. How is a polynomial represented using a singly linked list?
5. What is the time complexity of polynomial addition and multiplication?

RESULT

The implementation of polynomial addition and multiplication using a singly linked list was successfully completed. The results verified the correctness and efficiency of linked list-based polynomial operations using dynamic memory allocation.

EXP NO:7**BINARY EXPRESSION TREE WITH PREFIX/POSTFIX CONVERSION**

AIM: To Create a binary tree for a given simple arithmetic expression and find the prefix / postfix equivalent

LEARNING OUTCOMES

Learners will be able to construct a binary expression tree for a given arithmetic expression, represent operators and operands as tree nodes, generate the equivalent prefix and postfix expressions through tree traversals, understand the relationship between expression trees and expression notations, implement binary tree operations for expression conversion, and analyze the efficiency of tree-based expression processing.

DESCRIPTION

In a binary expression tree, each internal node represents an operator, while each leaf node represents an operand. The tree is constructed based on the precedence and associativity of operators. Once the tree is created, different tree traversals are used to obtain the equivalent expressions: **preorder traversal** generates the prefix expression, and **postorder traversal** generates the postfix expression.

ALGORITHM**Node Structure**

- Each node contains:
 1. data → operand or operator
 2. left → pointer to left child
 3. right → pointer to right child

 - Maintain **two stacks**:
 1. **Operators stack** → stores operators and parentheses
 2. **Operands stack** → stores tree nodes
1. Read fully parenthesized infix expression.

2. Call **INFIX TO EXPRESSION TREE** → get root node.
3. Initialize empty stacks: operators and operands.

Traverse the infix expression character by character:

- If character is '(' → push onto operators stack.
- If character is **operand** (alphanumeric) → create a node and push onto operands stack.
- If character is **operator** (+ - * /):

While top of operators stack has **higher or equal precedence** and is not '(':

- Pop operator from operators stack.
- Pop two nodes from operands stack.
- Make operator node as root, left child = first popped node, right child = second popped node.
- Push this operator node onto operands stack.

Push current operator onto operators stack.

- If character is ')':

While top of operators stack is not '(':

- Pop operator and create node as above.

Pop '(' from operators stack.

After traversal, while operators stack is not empty:

- Pop operator and create node as above.

The node remaining on operands stack is the **root** of the expression tree.

4. Print "Prefix expression: " and call **PREORDER(root)**.
 - Start at root node.
 - Print data.

- Recursively traverse left subtree.
- Recursively traverse right subtree.

5. Print "Postfix expression: " and call **POSTORDER(root)**.

Start at root node.

Recursively traverse left subtree.

Recursively traverse right subtree.

Print data.

PROGRAM

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>
// Node structure for binary expression tree
typedef struct Node {
    char data;
    struct Node *left, *right;
} Node;
// Stack for tree nodes
typedef struct Stack {
    Node* items[100];
    int top;
} Stack;
// Stack operations
void initStack(Stack *s) { s->top = -1; }
int isEmpty(Stack *s) { return s->top == -1; }
void push(Stack *s, Node* node) { s->items[++(s->top)] = node; }
Node* pop(Stack *s) { return s->items[(s->top)--]; }
Node* peek(Stack *s) { return s->items[s->top]; }
// Function to create a new tree node
```

```
Node* createNode(char data) {
    Node* node = (Node*)malloc(sizeof(Node));
    node->data = data;
    node->left = node->right = NULL;
    return node;
}

// Function to return precedence of operators
int precedence(char op) {
    if (op == '+' || op == '-') return 1;
    if (op == '*' || op == '/') return 2;
    return 0;
}

// Check if character is operator
int isOperator(char ch) {
    return ch == '+' || ch == '-' || ch == '*' || ch == '/';
}

// Convert infix to binary expression tree
Node* infixToExprTree(char* exp) {
    Stack operators, operands;
    initStack(&operators);
    initStack(&operands);
    int i;
    for (i = 0; i < strlen(exp); i++) {
        char ch = exp[i];
        if (ch == ' ' || ch == '\t') continue;
        if (ch == '(') {
            push(&operators, createNode(ch));
        } else if (isalnum(ch)) {
            push(&operands, createNode(ch));
        } else if (isOperator(ch)) {
            while (!isEmpty(&operators) && peek(&operators)->data != '(' &&
```

```
    precedence(peek(&operators)->data) >= precedence(ch)) {
        Node* op = pop(&operators);
        op->right = pop(&operands);
        op->left = pop(&operands);
        push(&operands, op);
    }
    push(&operators, createNode(ch));
} else if (ch == ')') {
    while (!isEmpty(&operators) && peek(&operators)->data != '(') {
        Node* op = pop(&operators);
        op->right = pop(&operands);
        op->left = pop(&operands);
        push(&operands, op);
    }
    pop(&operators); // remove '('
}
}
while (!isEmpty(&operators)) {
    Node* op = pop(&operators);
    op->right = pop(&operands);
    op->left = pop(&operands);
    push(&operands, op);
}
return pop(&operands); // root of expression tree
}

// Preorder traversal for prefix
void preorder(Node* root) {
    if (root != NULL) {
        printf("%c", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}
```

```
    }  
}  
// Postorder traversal for postfix  
void postorder(Node* root) {  
    if (root != NULL) {  
        postorder(root->left);  
        postorder(root->right);  
        printf("%c", root->data);  
    }  
}  
// Main function  
int main() {  
    char exp[100];  
    printf("Enter a fully parenthesized arithmetic expression: ");  
    fgets(exp, 100, stdin);  
    exp[strcspn(exp, "\n")] = 0; // remove newline  
    Node* root = infixToExprTree(exp);  
    printf("Prefix expression: ");  
    preorder(root);  
    printf("\n");  
    printf("Postfix expression: ");  
    postorder(root);  
    printf("\n");  
    return 0;  
}
```

OUTPUT

Example1:

(a+(b*c))

Prefix expression: +a*bc

Postfix expression: abc*+

Example2:

$((a+b)*(c-d))$

Prefix expression: $*+ab-cd$

Postfix expression: $ab+cd-*$

MODEL VIVA QUESTIONS

1. What is a binary expression tree?
2. Which tree traversal is used to generate a prefix expression?
3. What is the time complexity of traversing a binary expression tree?
4. What is an infix expression?
5. What is a prefix expression?
6. What is a postfix expression?

RESULT

The binary expression tree for the given arithmetic expression was successfully constructed. The equivalent prefix and postfix expressions were correctly generated using preorder and postorder tree traversals, respectively. The experiment verified the correctness of binary tree construction and demonstrated the effective use of tree traversal techniques for expression conversion.

EXP NO:8**BINARY SEARCH TREE**

AIM: Implement a dictionary of word-meaning pairs using binary search trees

LEARNING OUTCOMES

Learners will be able to implement a dictionary of word–meaning pairs using a Binary Search Tree (BST), perform insertion, searching, deletion, and traversal operations efficiently, organize dictionary entries in sorted order based on words, and analyze the efficiency of BST operations for dictionary management.

DESCRIPTION

This experiment demonstrates the implementation of a dictionary using a Binary Search Tree (BST), where each node stores a word and its corresponding meaning. The words are organized in alphabetical order according to the BST property, enabling efficient searching, insertion, and deletion of entries. Traversal techniques such as inorder traversal are used to display the dictionary in sorted order. This experiment helps in understanding the application of BSTs in implementin

ALGORITHM

1. Start
2. Initialize
 - Set root $\leftarrow$ NULL.
3. Repeat the following until user chooses Exit:
 - 3.1 Display menu
 1. Insert Word
 2. Search Word
 3. Display Dictionary
 4. Exit

3.2 Read user's choice

4. If choice = 1 (Insert Word)

1. Read word.
2. Read meaning.
3. Call insert(root, word, meaning) and update root.

Insert Operation

1. If root = NULL, create a new node and return it.
2. Compare word with root->word:
 - If word < root->word, insert into left subtree.
 - If word > root->word, insert into right subtree.
 - If equal, display "Word already exists."
3. Return updated root.

5. If choice = 2 (Search Word)

1. Read word to search.
2. Call search(root, word).

Search Operation

1. If current node is NULL, return NULL.
2. Compare word with current node's word:
 - If equal → return node.
 - If smaller → search left subtree.
 - If larger → search right subtree.
3. If result is found, display meaning; otherwise display "Word not found."

6. If choice = 3 (Display Dictionary)

1. Call inorder(root).

Inorder Traversal

1. Traverse left subtree.
2. Print current node's word and meaning.
3. Traverse right subtree.

7. If choice = 4

- Stop program.

8. End

PROGRAM

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
// Node structure
typedef struct Node {
    char word[50];
    char meaning[200];
    struct Node *left, *right;
} Node;
// Function to create a new node
Node* createNode(char *word, char *meaning) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    strcpy(newNode->word, word);
    strcpy(newNode->meaning, meaning);
    newNode->left = newNode->right = NULL;
    return newNode;
}
// Insert node into BST (word alphabetical order)
Node* insert(Node* root, char *word, char *meaning) {
    if (root == NULL)
```

```
        return createNode(word, meaning);
    int cmp = strcmp(word, root->word);
    if (cmp < 0)
        root->left = insert(root->left, word, meaning);
    else if (cmp > 0)
        root->right = insert(root->right, word, meaning);
    else
        printf("Word already exists in dictionary!\n");

    return root;
}

// Search for a word in dictionary
Node* search(Node* root, char *word) {
    if (root == NULL)
        return NULL;
    int cmp = strcmp(word, root->word);
    if (cmp == 0)
        return root;
    else if (cmp < 0)
        return search(root->left, word);
    else
        return search(root->right, word);
}

// Display dictionary in alphabetical order
void inorder(Node* root) {
    if (root != NULL) {
        inorder(root->left);
        printf("%s : %s\n", root->word, root->meaning);
        inorder(root->right);
    }
}
```

```
int main() {
    Node* root = NULL;
    int choice;
    char word[50], meaning[200];
    while (1) {
        printf("\n--- DICTIONARY MENU (BST) ---\n");
        printf("1. Insert Word\n2. Search Word\n3. Display Dictionary\n4. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);
        getchar(); // remove newline
        switch (choice) {
            case 1:
                printf("Enter word: ");
                fgets(word, 50, stdin);
                word[strcspn(word, "\n")] = 0;
                printf("Enter meaning: ");
                fgets(meaning, 200, stdin);
                meaning[strcspn(meaning, "\n")] = 0;
                root = insert(root, word, meaning);
                break;
            case 2:
                printf("Enter word to search: ");
                fgets(word, 50, stdin);
                word[strcspn(word, "\n")] = 0;
                Node* result = search(root, word);
                if (result)
                    printf("Meaning: %s\n", result->meaning);
                else
                    printf("Word not found in dictionary.\n");
                break;
            case 3:
```

```
        printf("\n--- Dictionary Contents (Alphabetical Order) ---\n");
        inorder(root);
        break;

    case 4:
        return 0;
    default:
        printf("Invalid choice!\n");
    }
}
}
```

OUTPUT

--- DICTIONARY MENU (BST) ---

1. Insert Word
2. Search Word
3. Display Dictionary
4. Exit

Enter choice: 1

Enter word: apple

Enter meaning: A fruit

Enter choice: 1

Enter word: cat

Enter meaning: A small animal

Enter choice: 3

--- Dictionary Contents (Alphabetical Order) ---

apple : A fruit

cat : A small animal

Enter choice: 2

Enter word: apple

MODEL VIVA QUESTIONS

1. What is a Binary Search Tree (BST)?

2. What is the BST property?
3. Why is a BST suitable for implementing a dictionary?
4. What information is stored in each node of the dictionary BST?
5. How is a new word inserted into the BST?

RESULT

The dictionary of word–meaning pairs was successfully implemented using a Binary Search Tree. The program correctly performed insertion, searching, deletion, and traversal operations while maintaining the BST property. The dictionary entries were displayed in alphabetical order, demonstrating the efficiency and correctness of BST-based dictionary management.

EXP NO:9**SORTING ALGORITHMS**

AIM: Implement Bubble sort, Insertion Sort, selection sort, Quick Sort, and Merge Sort and Heap sort.

LEARNING OUTCOMES

Learners will be able to implement Bubble Sort, Insertion Sort, Selection Sort, Quick Sort, Merge Sort, and Heap Sort algorithms, compare their working principles and time complexities, analyze their performance for different input sizes, and select an appropriate sorting technique based on efficiency and application requirements.

DESCRIPTION**Bubble Sort:**

The algorithm repeatedly compares adjacent elements in the list and swaps them if they are in the wrong order. After each pass, the largest unsorted element "bubbles up" to its correct position at the end of the list. The process continues until the entire list is sorted. Bubble Sort is easy to understand and implement, making it suitable for learning the fundamentals of sorting algorithms, although it is less efficient for large datasets due to its quadratic time complexity.

Insertion Sort:

The algorithm divides the list into a sorted and an unsorted part. Starting from the second element, each element is selected and inserted into its correct position within the already sorted portion by shifting larger elements one position to the right. This process continues until all elements are placed in their proper positions, resulting in a fully sorted list. Insertion Sort is efficient for small datasets and performs particularly well when the input is already nearly sorted.

Selection Sort:

The algorithm divides the array into sorted and unsorted portions. In each pass, it selects the smallest (or largest) element from the unsorted portion and swaps it with the first unsorted element. This process is repeated until all elements are placed in their correct positions. Selection Sort performs a fixed number of comparisons regardless of the initial arrangement of the data and is suitable for small datasets due to its simplicity.

Quick Sort:

The algorithm selects a **pivot** element and partitions the array into two subarrays such that elements smaller

than the pivot are placed to its left and larger elements to its right. The same process is then recursively applied to the left and right subarrays until the entire array is sorted. Quick Sort is widely used because of its excellent average-case performance and efficient use of memory for large datasets.

Merge Sort:

The algorithm recursively divides the array into smaller subarrays until each subarray contains a single element. These subarrays are then merged in a sorted manner to produce the final sorted array. Since Merge Sort consistently performs well regardless of the input order, it is widely used for sorting large datasets and external sorting applications. However, it requires additional memory to store temporary subarrays during the merging process.

Heap sort:

Initially, the given array is converted into a max heap (for ascending order), where the largest element is placed at the root. The root element is then swapped with the last element of the heap, the heap size is reduced, and the heap property is restored using the heapify operation. This process is repeated until all elements are sorted. Heap Sort provides consistent $O(n \log n)$ time complexity and performs sorting in place without requiring significant additional memory.

ALGORITHM**(A) BUBBLE SORT**

Step 1: Start

Step 2: Read n (number of elements)

Step 3: Read array $A[0 \dots n-1]$

Step 4: Repeat for $i = 0$ to $n - 2$

- a. Set swapped = 0
- b. Repeat for $j = 0$ to $n - i - 2$
 - i. If $A[j] > A[j+1]$
 - Swap $A[j]$ and $A[j+1]$
 - Set swapped = 1
- c. If swapped == 0, break

Step 5: Display sorted array

Step 6: Stop

(B) INSERTION SORT

Step 1: Start

Step 2: Read n (number of elements)

Step 3: Read array $A[0 \dots n-1]$

Step 4: Repeat for $i = 1$ to $n - 1$

- a. Set $\text{key} = A[i]$
- b. Set $j = i - 1$
- c. While $j \geq 0$ and $A[j] > \text{key}$
 - i. Set $A[j + 1] = A[j]$
 - ii. Decrement j
- d. Set $A[j + 1] = \text{key}$

Step 5: Display sorted array

Step 6: Stop

(C)SELECTION SORT

Step 1: Start

Step 2: Read n (number of elements)

Step 3: Read array $A[0 \dots n-1]$

Step 4: Repeat for $i = 0$ to $n - 2$

- a. Set $\text{min_index} = i$
- b. Repeat for $j = i + 1$ to $n - 1$
 - i. If $A[j] < A[\text{min_index}]$, set $\text{min_index} = j$
- c. Swap $A[i]$ and $A[\text{min_index}]$

Step 5: Display sorted array

Step 6: Stop

(D)QUICK SORT

Step 1: Start

Step 2: Read n (number of elements)

Step 3: Read array $A[0 \dots n-1]$

Step 4: Call $\text{QUICK_SORT}(A, \text{low} = 0, \text{high} = n-1)$

Procedure: QUICK_SORT($A, \text{low}, \text{high}$)

1. If $\text{low} < \text{high}$ then
 - a. Call $\text{PARTITION}(A, \text{low}, \text{high}) \rightarrow \text{returns pivot_index}$
 - b. Call $\text{QUICK_SORT}(A, \text{low}, \text{pivot_index} - 1)$
 - c. Call $\text{QUICK_SORT}(A, \text{pivot_index} + 1, \text{high})$
2. End

Procedure: PARTITION($A, \text{low}, \text{high}$)

1. Set $\text{pivot} = A[\text{high}]$
2. Set $i = \text{low} - 1$
3. Repeat for $j = \text{low}$ to $\text{high} - 1$
 - a. If $A[j] \leq \text{pivot}$
 - i. Increment i ($i = i + 1$)
 - ii. Swap $A[i]$ and $A[j]$
4. Swap $A[i + 1]$ and $A[\text{high}]$ (place pivot at correct position)
5. Return $i + 1$ (pivot index)

Step 5: Display sorted array

Step 6: Stop

(E)MERGE SORT

Step 1: Start

Step 2: Read n (number of elements) and array $A[0 \dots n-1]$

Step 3: Call $\text{MERGE_SORT}(A, \text{low} = 0, \text{high} = n-1)$

Procedure: MERGE_SORT(A, low, high)

1. If $\text{low} < \text{high}$ then:
 - a. Find middle: $\text{mid} = (\text{low} + \text{high}) / 2$
 - b. Call $\text{MERGE_SORT}(A, \text{low}, \text{mid})$
 - c. Call $\text{MERGE_SORT}(A, \text{mid} + 1, \text{high})$
 - d. Call $\text{MERGE}(A, \text{low}, \text{mid}, \text{high})$ to merge two halves

Procedure: MERGE(A, low, mid, high)

1. Create temporary arrays $L[0 \dots \text{mid}-\text{low}]$ and $R[0 \dots \text{high}-\text{mid}-1]$
2. Copy elements from $A[\text{low} \dots \text{mid}]$ into L
3. Copy elements from $A[\text{mid}+1 \dots \text{high}]$ into R
4. Initialize $i = 0, j = 0, k = \text{low}$
5. While $i < \text{size of } L$ and $j < \text{size of } R$

If $L[i] \leq R[j]$

■ $A[k] = L[i]$

■ $i = i + 1$

Else

■ $A[k] = R[j]$

■ $j = j + 1$

$k = k + 1$

6. Copy remaining elements of L (if any) to A
7. Copy remaining elements of R (if any) to A
8. End

Step 4: Display sorted array

Step 5: Stop

(F)HEAP SORT

Step 1: Start

Step 2: Read n (number of elements) and array $A[0 \dots n-1]$

Step 3: Build a **max heap** from the array

- For $i = n/2 - 1$ down to 0

Call HEAPIFY(A, n, i)

Procedure: HEAPIFY(A, n, i)

1. Set $\text{largest} = i$
2. $\text{left} = 2*i + 1$
3. $\text{right} = 2*i + 2$
4. If $\text{left} < n$ and $A[\text{left}] > A[\text{largest}]$, set $\text{largest} = \text{left}$
5. If $\text{right} < n$ and $A[\text{right}] > A[\text{largest}]$, set $\text{largest} = \text{right}$
6. If $\text{largest} \neq i$

Swap $A[i]$ and $A[\text{largest}]$

Call HEAPIFY($A, n, \text{largest}$)

Step 4: Extract elements from heap one by one

- For $i = n - 1$ down to 1
 1. Swap $A[0]$ and $A[i]$
 2. Call HEAPIFY($A, i, 0$)

Step 5: Display sorted array

Step 6: Stop

PROGRAM**(A)BUBBLE SORT**

```
#include <stdio.h>

int main() {
    int n, i, j, temp;
    int a[100];

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }

    // Bubble Sort
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (a[j] > a[j + 1]) {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }

    printf("Sorted array:\n");
    for (i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }

    return 0;
}
```

(B)INSERTION SORT

```
#include <stdio.h>
```

```
int main() {
    int n, i, j, key;
    int a[100];
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }
    // Insertion Sort
    for (i = 1; i < n; i++) {
        key = a[i];
        j = i - 1;
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }
    printf("Sorted array:\n");
    for (i = 0; i < n; i++) {
        printf("%d ", a[i]);
    }
    return 0;
}
```

(C)SELECTION SORT

```
#include <stdio.h>

int main() {
    int n, i, j, min_index, temp;
    int a[100];
    printf("Enter number of elements: ");
```

```
scanf("%d", &n);
printf("Enter %d elements:\n", n);
for (i = 0; i < n; i++) {
    scanf("%d", &a[i]);
}
// Selection Sort
for (i = 0; i < n - 1; i++) {
    min_index = i;

    for (j = i + 1; j < n; j++) {
        if (a[j] < a[min_index]) {
            min_index = j;
        }
    }
    // Swap a[i] and a[min_index]
    temp = a[i];
    a[i] = a[min_index];
    a[min_index] = temp;
}
printf("Sorted array:\n");
for (i = 0; i < n; i++) {
    printf("%d ", a[i]);
}
return 0;
}
```

(D)QUICK SORT

```
#include <stdio.h>
// Function to swap two elements
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
```

```
*b = temp;
}
// Partition function
int partition(int a[], int low, int high) {
    int pivot = a[high]; // Choose last element as pivot
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (a[j] <= pivot) {
            i++;
            swap(&a[i], &a[j]);
        }
    }
    swap(&a[i + 1], &a[high]);
    return i + 1;
}
// Quick Sort function
void quickSort(int a[], int low, int high) {
    if (low < high) {
        int pi = partition(a, low, high);
        quickSort(a, low, pi - 1);
        quickSort(a, pi + 1, high);
    }
}
// Main function
int main() {
    int n, i;
    int a[100];
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d elements:\n", n);
    for (i = 0; i < n; i++) {
```

```
    scanf("%d", &a[i]);
}
quickSort(a, 0, n - 1);

printf("Sorted array:\n");
for (i = 0; i < n; i++) {
    printf("%d ", a[i]);
}
return 0;
}
```

(F)HEAP SORT

```
#include <stdio.h>
// Function to swap two elements
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}
// Heapify function to maintain max-heap property
void heapify(int a[], int n, int i) {
    int largest = i;    // Initialize largest as root
    int left = 2 * i + 1; // Left child index
    int right = 2 * i + 2; // Right child index
    if (left < n && a[left] > a[largest])
        largest = left;
    if (right < n && a[right] > a[largest])
        largest = right;
    if (largest != i) {
        swap(&a[i], &a[largest]);
        heapify(a, n, largest); // Recursively heapify the affected sub-tree
    }
}
```

```
    }  
}  
// Heap Sort function  
void heapSort(int a[], int n) {  
    // Build max heap  
    for (int i = n / 2 - 1; i >= 0; i--)  
        heapify(a, n, i);  
    // Extract elements from heap  
    for (int i = n - 1; i >= 0; i--) {  
        swap(&a[0], &a[i]); // Move current root to end  
        heapify(a, i, 0); // Heapify reduced heap  
    }  
}  
  
int main() {  
    int n;  
    int a[100];  
    printf("Enter number of elements: ");  
    scanf("%d", &n);  
    printf("Enter %d elements:\n", n);  
    for (int i = 0; i < n; i++) {  
        scanf("%d", &a[i]);  
    }  
    heapSort(a, n);  
    printf("Sorted array:\n");  
    for (int i = 0; i < n; i++) {  
        printf("%d ", a[i]);  
    }  
    return 0;  
}
```

OUTPUT**(A)BUBBLE SORT**

Enter number of elements: 5

Enter 5 elements:

34 12 25 9 1

Sorted array:

1 9 12 25 34

(B)INSERTION SORT

Enter number of elements: 5

Enter 5 elements:

8 3 6 2 5

Sorted array:

2 3 5 6 8

(C)SELECTION SORT

Enter number of elements: 6

Enter 6 elements:

29 10 14 37 13 5

Sorted array:

5 10 13 14 29 37

(D)QUICK SORT

Enter number of elements: 6

Enter 6 elements:

29 10 14 37 13 5

Sorted array:

5 10 13 14 29 37

(E)MERGE SORT

Enter number of elements: 6

Enter 6 elements:

29 10 14 37 13 5

Sorted array:

5 10 13 14 29 37

(F)HEAP SORT

Enter number of elements: 6

Enter 6 elements:

29 10 14 37 13 5

Sorted array:

5 10 13 14 29 37

MODEL VIVA QUESTIONS

1. What is sorting?
2. Why is sorting important?
3. What is the difference between internal and external sorting?
4. What is the time complexity of Heap Sort?
5. Which sorting algorithm requires additional memory?
6. Which sorting algorithm is suitable for small datasets?
7. What are the applications of sorting algorithms?

RESULT

The implementation of Bubble Sort, Insertion Sort, Selection Sort, Quick Sort, Merge Sort, and Heap Sort was successfully completed. Each algorithm correctly sorted the given set of elements in ascending order. The experiment verified the correctness of the sorting techniques and demonstrated the differences in their execution efficiency, time complexity, and suitability for various applications.

EXP NO:10A**BINARY SEARCH**

AIM: Given an array of sorted items, implement an efficient algorithm to search for specific item in the array.

LEARNING OUTCOMES

Learners will be able to implement the Binary Search algorithm, perform efficient searching on a sorted array, understand the divide-and-conquer approach used in searching, analyze the time and space complexity of Binary Search, and compare its performance with linear search.

DESCRIPTION

The algorithm repeatedly compares the target element with the middle element of the array. If the target matches the middle element, the search is successful. If the target is smaller, the search continues in the left half; otherwise, it continues in the right half. This process is repeated until the element is found or the search interval becomes empty. Binary Search significantly reduces the number of comparisons, making it much faster than linear search for large sorted datasets.

ALGORITHM

1. Start
2. Initialize Pointers low = 0 and high = n - 1
3. Repeat While low $\leq$ high
 - a. Calculate middle index , mid = low + (high - low) / 2
 - b. Compare arr[mid] with x:
 - i. If arr[mid] == x $\rightarrow$ return mid (item found)
 - ii. Else if arr[mid] < x $\rightarrow$ search right half, ie low = mid + 1
 - iii. Else if arr[mid] > x $\rightarrow$ search left half, ie high = mid - 1
4. If loop ends
 - a. If low > high $\rightarrow$ return -1 (item not found)
5. end

PROGRAM

```
#include <stdio.h>
```

```
int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] == key)
            return mid; // item found
        else if (arr[mid] < key)
            low = mid + 1; // search right half
        else
            high = mid - 1; // search left half
    }
    return -1; // item not found
}

int main() {
    int n, key;
    printf("Enter size of array: ");
    scanf("%d", &n);
    int arr[n];
    printf("Enter %d elements in sorted order:\n", n);
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);
    printf("Enter item to search: ");
    scanf("%d", &key);
    int index = binarySearch(arr, n, key);
    if (index != -1)
        printf("Item %d found at index %d\n", key, index);
    else
        printf("Item %d not found in array\n", key);
    return 0;
}
```

OUTPUT

Enter size of array: 6

Enter 6 elements in sorted order:

1 3 5 7 9 11

Enter item to search: 7

Item 7 found at index 3

MODEL VIVA QUESTIONS

1. How does Binary Search differ from Linear Search?
2. What are the advantages of Binary Search?
3. What are the limitations of Binary Search?

RESULT

The Binary Search algorithm was successfully implemented and executed. The program correctly searched for the specified element in the sorted array and displayed whether the element was found along with its position. The experiment verified the correctness and efficiency of Binary Search for searching elements in sorted data.

EXP NO:10B**LINEAR SEARCH**

AIM: Given an array of sorted items, implement an linear search algorithm to search for specific item in the array.

LEARNING OUTCOMES

Learners will be able to implement the Linear Search algorithm, perform sequential searching of elements in an array, understand the working principle of linear search, analyze its time and space complexity, and compare its performance with Binary Search.

DESCRIPTION

The algorithm searches for a given element by comparing it sequentially with each element in the array, starting from the first element until the desired element is found or the end of the array is reached. Unlike Binary Search, Linear Search does not require the data to be sorted, making it suitable for searching in small or unsorted datasets. Although simple to implement, it becomes less efficient for large datasets due to its sequential nature.

ALGORITHM

Step 1: Start

Step 2: Read n (number of elements) and array A[0...n-1]

Step 3: Read the KEY to be searched

Step 4: Set found = false

Step 5: Repeat for i = 0 to n - 1

- a. If A[i] == KEY then
 - i. Set found = true
 - ii. Print "KEY found at position i+1"
 - iii. Exit loop

Step 6: If found == false then

Print “KEY not found”

Step 7: Stop

PROGRAM

```
#include <stdio.h>

int main() {
    int n, key, found = 0;
    int a[100];
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter %d elements:\n", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &a[i]);
    }
    printf("Enter the element to search: ");
    scanf("%d", &key);
    // Linear Search
    for (int i = 0; i < n; i++) {
        if (a[i] == key) {
            printf("%d found at position %d\n", key, i + 1);
            found = 1;
            break;
        }
    }
    if (!found) {
        printf("%d not found in the array\n", key);
    }
    return 0;
}
```

OUTPUT

Enter number of elements: 5

Enter 5 elements:

10 25 30 15 40

Enter the element to search: 30

30 found at position 3

MODEL VIVA QUESTIONS

1. What is Linear Search?
2. How does Linear Search work?
3. What are the advantages of Linear Search?
4. Can Linear Search be used with linked lists?

RESULT

The Linear Search algorithm was successfully implemented and executed. The experiment verified the correctness of the algorithm and demonstrated its effectiveness for searching elements in small or unsorted datasets.

EXP NO:11**BFS AND DFS**

AIM: To write a c program to implement BFS and DFS

LEARNING OUTCOMES

Learners will be able to represent graphs using suitable data structures, implement Breadth-First Search (BFS) and Depth-First Search (DFS) traversal algorithms, understand the working principles of graph traversal techniques, analyze their time and space complexities, and apply them to solve graph-related problems such as path finding, connectivity, and cycle detection.

DESCRIPTION**Breadth-First Search (BFS)**

Breadth-First Search (BFS) is a graph traversal algorithm that visits the vertices of a graph level by level. It starts from a selected source (starting) vertex, visits all its adjacent (neighboring) vertices first, and then moves to the next level of vertices. BFS uses a Queue (FIFO – First In, First Out) to keep track of the vertices that need to be visited.

Depth-First Search (DFS)

Depth-First Search (DFS) is a graph traversal algorithm that explores a graph by visiting a source (starting) vertex and then moving as deep as possible along each branch before backtracking. It uses a Stack (LIFO – Last In, First Out) or recursion to keep track of the vertices to be visited. A visited array is maintained to ensure that each vertex is visited only once.

ALGORITHM**Breadth-First Search (BFS)**

1. Start with the source vertex and mark it as visited.
2. Insert the source vertex into the queue.

3. While the queue is not empty:
 Remove (dequeue) the front vertex from the queue.
 Visit and display the vertex.
 For each unvisited adjacent vertex:
 - Mark it as visited.
 - Insert (enqueue) it into the queue.
4. Repeat the process until the queue becomes empty.
5. End

Depth-First Search (DFS)

1. Start from the source vertex and mark it as **visited**.
2. Visit and display the current vertex.
3. Select an unvisited adjacent vertex.
4. Mark the adjacent vertex as **visited** and recursively visit it (or push it onto the stack).
5. If a vertex has no unvisited adjacent vertices, **backtrack** to the previous vertex.
6. Repeat the process until all vertices in the graph are visited.
7. End

PROGRAM

(A) Breadth-First Search (BFS)

```
#include <stdio.h>

#define MAX 20

int graph[MAX][MAX], visited[MAX], queue[MAX];
int front = -1, rear = -1, n;

void enqueue(int item)
{
    if (rear == MAX - 1)
        return;
    if (front == -1)
        front = 0;
    queue[++rear] = item;
}
```

```
int dequeue()
{
    if (front == -1 || front > rear)
        return -1;
    return queue[front++];
}

void bfs(int start)
{
    int i, vertex;

    for (i = 0; i < n; i++)
        visited[i] = 0;

    visited[start] = 1;
    enqueue(start);

    printf("BFS Traversal: ");

    while (front != -1 && front <= rear)
    {
        vertex = dequeue();
        printf("%d ", vertex);

        for (i = 0; i < n; i++)
        {
            if (graph[vertex][i] == 1 && !visited[i])
            {
                visited[i] = 1;
                enqueue(i);
            }
        }
    }
}

int main()
{
    int i, j, start;

    printf("Enter the number of vertices: ");
    scanf("%d", &n);

    printf("Enter the adjacency matrix:\n");
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++)
        {
```

```
        scanf("%d", &graph[i][j]);
    }
}

printf("Enter the starting vertex (0 to %d): ", n - 1);
scanf("%d", &start);

bfs(start);

return 0;
}
```

(B)Depth-First Search (DFS)

```
#include <stdio.h>

#define MAX 20

int graph[MAX][MAX], visited[MAX], n;

void dfs(int vertex){
    int i;
    visited[vertex] = 1;
    printf("%d ", vertex);
    for (i = 0; i < n; i++)
    {
        if (graph[vertex][i] == 1 && !visited[i])
        {
            dfs(i);
        }
    }
}

int main()
{
    int i, j, start;
```

```
printf("Enter the number of vertices: ");  
  
scanf("%d", &n);  
  
printf("Enter the adjacency matrix:\n");  
  
for (i = 0; i < n; i++)  
{  
    for (j = 0; j < n; j++)  
    {  
        scanf("%d", &graph[i][j]);  
    }  
}  
  
for (i = 0; i < n; i++)  
    visited[i] = 0;  
  
printf("Enter the starting vertex (0 to %d): ", n - 1);  
  
scanf("%d", &start);  
  
printf("DFS Traversal: ");  
  
dfs(start);  
  
return 0;  
}
```

OUTPUT**(A) Breadth-First Search (BFS)**

Enter the number of vertices: 5

Enter the adjacency matrix:

0 1 1 0 0

1 0 0 1 1

1 0 0 0 0

0 1 0 0 0

0 1 0 0 0

Enter the starting vertex: 0

BFS Traversal: 0 1 2 3 4

(B)Depth-First Search (DFS)

Enter the number of vertices: 5

Enter the adjacency matrix:

0 1 1 0 0

1 0 0 1 1

1 0 0 0 0

0 1 0 0 0

0 1 0 0 0

Enter the starting vertex: 0

DFS Traversal: 0 1 3 4 2

MODEL VIVA QUESTIONS

1. What is Breadth-First Search (BFS)?
2. What is graph traversal?
3. Which data structure is used in BFS?
4. Why is a queue used in BFS?

RESULT

The implementation of Breadth-First Search (BFS) and Depth-First Search (DFS) was successfully completed. The program correctly traversed all the vertices of the graph using both algorithms and displayed the traversal order. The experiment verified the correctness of graph representation and demonstrated the efficiency of BFS and DFS in exploring graph structures.

EXP NO:12**IMPLEMENTATION OF A BASIC MEMORY ALLOCATOR WITH GARBAGE COLLECTION**

AIM: Simulation of a basic memory allocator and garbage collector using doubly linked list

LEARNING OUTCOMES

Learners will be able to implement a basic memory allocator and garbage collector using a doubly linked list, manage dynamic memory allocation and deallocation, understand memory block organization, simulate garbage collection techniques, and analyze the efficiency of memory management using linked list data structures.

DESCRIPTION

Each node in the doubly linked list represents a memory block containing information such as block size, allocation status (allocated or free), and pointers to the previous and next memory blocks. The memory allocator assigns available memory blocks to user requests, while the garbage collector identifies and reclaims unused memory blocks to make them available for future allocations. The doubly linked list allows efficient traversal in both forward and backward directions, making memory allocation, deallocation, merging of adjacent free blocks, and garbage collection more efficient.

ALGORITHM

1. Start.
2. Create a doubly linked list to represent memory blocks.
3. Initialize each node with the block ID, block size, allocation status (free), and pointers to the previous and next nodes.
4. Display the menu with the following options:
 - Allocate Memory
 - Deallocate Memory
 - Garbage Collection

Display Memory Blocks

Exit

5. Read the user's choice.
6. If Allocate Memory is selected:
 - Read the required memory size.
 - Traverse the linked list to find the first free block with sufficient size.
 - If found, mark the block as allocated.
 - Otherwise, display "Memory Allocation Failed".
7. If Deallocate Memory is selected:
 - Read the block ID.
 - Traverse the linked list and locate the corresponding block.
 - Mark the block as free.
8. If Garbage Collection is selected:
 - Traverse the linked list.
 - Identify adjacent free memory blocks.
 - Merge adjacent free blocks into a single larger block.
 - Update the previous and next pointers accordingly.
9. If Display Memory Blocks is selected:
 - Traverse the linked list.
 - Display the block ID, block size, and allocation status of each memory block.
10. Repeat Steps 5–9 until the user selects Exit.
11. Stop.

PROGRAM

```
#include <stdio.h>
#include <stdlib.h>
struct Node
{
    int id;
    int size;
    int allocated;    // 1 = Allocated, 0 = Free
    struct Node *prev;
```

```
    struct Node *next;
};
struct Node *head = NULL;
// Create a memory block
void createBlock(int id, int size)
{
    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));

    newNode->id = id;
    newNode->size = size;
    newNode->allocated = 0;
    newNode->prev = NULL;
    newNode->next = NULL;
    if (head == NULL)
    {
        head = newNode;
        return;
    }
    struct Node *temp = head;
    while (temp->next != NULL)
        temp = temp->next;
    temp->next = newNode;
    newNode->prev = temp;
}
// Allocate memory
void allocateMemory(int reqSize)
{
    struct Node *temp = head;
    while (temp != NULL)
    {
        if (temp->allocated == 0 && temp->size >= reqSize)
```

```
{
    temp->allocated = 1;
    printf("\nMemory Allocated in Block %d\n", temp->id);
    return;
}
temp = temp->next;
}
printf("\nMemory Allocation Failed!\n");
}
// Deallocate memory
void deallocateMemory(int id)
{
    struct Node *temp = head;
    while (temp != NULL)
    {
        if (temp->id == id)
        {
            temp->allocated = 0;
            printf("\nMemory Deallocated from Block %d\n", id);
            return;
        }
        temp = temp->next;
    }
    printf("\nInvalid Block ID!\n");
}
// Garbage Collection (Merge Adjacent Free Blocks)
void garbageCollector()
{
    struct Node *temp = head;
    while (temp != NULL && temp->next != NULL)
    {
```

```
if (temp->allocated == 0 && temp->next->allocated == 0)
{
    struct Node *del = temp->next;
    temp->size += del->size;
    temp->next = del->next;

    if (del->next != NULL)
        del->next->prev = temp;
    free(del);
}
else
{
    temp = temp->next;
}
}
printf("\nGarbage Collection Completed.\n");
}
// Display memory blocks
void display()
{
    struct Node *temp = head;
    printf("\n-----\n");
    printf("Block\tSize\tStatus\n");
    printf("-----\n");
    while (temp != NULL)
    {
        printf("%d\t%d\t%s\n",
            temp->id,
            temp->size,
            temp->allocated ? "Allocated" : "Free");
    }
}
```

```
    temp = temp->next;
}
printf("-----\n");
}
int main()
{
    int n, i, size, choice, reqSize, id;

    printf("Enter Number of Memory Blocks: ");
    scanf("%d", &n);
    for (i = 1; i <= n; i++)
    {
        printf("Enter Size of Block %d: ", i);
        scanf("%d", &size);
        createBlock(i, size);
    }
    do
    {
        printf("\n===== MEMORY MANAGEMENT =====\n");
        printf("1. Allocate Memory\n");
        printf("2. Deallocate Memory\n");
        printf("3. Garbage Collection\n");
        printf("4. Display Memory Blocks\n");
        printf("5. Exit\n");
        printf("Enter Choice: ");
        scanf("%d", &choice);
        switch (choice)
        {
            case 1:
                printf("Enter Required Memory Size: ");
                scanf("%d", &reqSize);
```

```
        allocateMemory(reqSize);
        break;
    case 2:
        printf("Enter Block ID to Deallocate: ");
        scanf("%d", &id);
        deallocateMemory(id);
        break;
    case 3:
        garbageCollector();
        break;
    case 4:
        display();
        break;
    case 5:
        printf("\nProgram Terminated.\n");
        break;
    default:
        printf("\nInvalid Choice!\n");
    }
} while (choice != 5);
return 0;
}
```

OUTPUT

Enter Number of Memory Blocks: 3

Enter Size of Block 1: 100

Enter Size of Block 2: 200

Enter Size of Block 3: 150

===== MEMORY MANAGEMENT =====

1. Allocate Memory

2. Deallocate Memory

3. Garbage Collection

4. Display Memory Blocks

5. Exit

Enter Choice: 1

Enter Required Memory Size: 120

Memory Allocated in Block 2

===== MEMORY MANAGEMENT =====

Enter Choice: 4

Block Size Status

1	100	Free
2	200	Allocated
3	150	Free

MODEL VIVA QUESTIONS

1. What is dynamic memory allocation?
2. What is a memory allocator?
3. What is garbage collection?
4. Why is a doubly linked list used in memory management?

RESULT

The basic memory allocator with garbage collection was successfully implemented using a doubly linked list. The program correctly performed memory allocation, deallocation, and garbage collection by merging adjacent free memory blocks. It efficiently managed memory and displayed the status of all memory blocks, demonstrating the effectiveness of linked list-based dynamic memory management.

